

Individual, Group and Organizational Learning in Software Development: A Learning Curve Analysis

ABSTRACT

Successfully developing software systems is difficult, and many software development efforts fail. Given the widespread incidence of software deployment problems and the proliferation of unreliable software, there is a need to examine whether individuals, groups and organizations learn from their experience in systems development. This study investigates whether the organizational learning curve is salient in software development. Prior research has found the existence of learning curves in manufacturing and service industries, but has not examined whether learning curves exist in knowledge work such as software development. Further, the research on organizational learning typically investigates whether there is learning at one level of analysis. This study examines whether learning occurs at the individual, group and organizational levels in systems development at a large telecommunications firm. Findings indicate that individuals' productivity in developing a system is positively affected by their prior experience with that system as well as their prior experience working on other systems. This provides evidence of individual level learning and of individuals' ability to effectively transfer their learning across systems. Further, findings support the existence of group level learning, indicating that groups' productivity in developing a system is affected by the group members' experience of working with one another on past projects. However, we find no evidence of learning at the organizational level across major releases of the systems. Our findings highlight the importance of differentiating learning at different levels of analysis in organizations, as there can be learning at some levels but not at other levels.

Key Words: Information Systems Development; Knowledge Work, Knowledge Workers, and Knowledge Networks; Organizational Learning.

Individual, Group and Organizational Learning in Software Development: A Learning Curve Analysis

I. INTRODUCTION

Although software systems have become critical to almost every aspect of modern life, systems development projects are frequently behind schedule and over budget, and once installed, the software often does not work reliably (Mann, 2002). Over the past few decades, the software industry has been assailed by numerous accounts of systems development failures as well as poor product quality delivered by software development organizations in both the commercial and government sectors (Kellner, Madachy, & Raffo, 1999). It is striking that these problems have persisted despite automated tools, advances in software development programming languages and methods, and formal education and training in computer science and information systems (Brooks, 1995). There are many potential reasons for the difficulties in software development. One explanation not often considered is that those producing the systems are not adequately learning from their experience. In this study, we investigate the question of whether organizations, groups and individuals engaging in software development actually learn from their experience and can transfer their knowledge to new systems projects.

A key challenge in studying learning in organizations is the difficulty in quantifying the extent of learning. There are many different definitions of organizational learning available in the literature, which have different implications for how learning should be measured. Some hold the perspective that organizational learning must include change, such that an organization can be said to learn when its actions have been modified as a result of new knowledge (Edmondson, 2002; Garvin, 2000), while others posit that learning need not result in observable changes in behavior (Huber, 1991). The disadvantage in studying learning by examining changes in knowledge is that a significant component of the knowledge that organizations acquire may be

tacit and not easily articulated (Nonaka, 1991). Moreover, the knowledge in organizations resides in multiple repositories (Levitt & March, 1988) and the operationalization of knowledge differs from one context to another. Changes in knowledge are thus difficult to measure and compare, whereas changes in behavior are more observable and able to be objectively measured.

The organizational learning curve approach adopts the perspective that organizational learning should entail changes in behavior. The organizational learning curve has been the subject of extensive study in many fields, like operations management and economics, and its presence has been widely documented in many industries (Argote, Beckman, & Epple, 1990; Pisano, Bohmer, & Edmondson, 2001). The basic principle underlying the learning curve is that production experience creates knowledge that improves productivity. By empirically examining whether productivity of the organization changes as organizations acquire experience, the learning curve investigates whether the unit cost of production decreases at a decreasing rate as organizations produce more of a product (Argote, 1999).

In this study, we examine whether the organizational learning curve is salient in software systems development. While prior research has found the existence of learning curves in manufacturing and service industries, it is not clear whether learning curves exist in knowledge work such as software development. Further, the research on organizational learning typically studies learning at one level of analysis. We seek to contribute to the information systems and organizational learning literatures by teasing out the extent to which learning occurs in software development work and the extent to which learning is attributable to individuals, groups or organizations. We extend the use of the learning curve to the knowledge work context, which has distinct characteristics from the more routinized manufacturing and scripted service industries.

In the next sections of this paper, we briefly review the literature on learning curves and consider how learning curves would apply in software systems development. We then develop

hypotheses concerning learning at the individual, group, and organizational levels and empirically examine our hypotheses in a field study of software systems development at a large telecommunications company. Finally, we present the results of our analysis, discuss their implications, and conclude by suggesting areas for further research.

II. RELATED LITERATURE & HYPOTHESES

Learning Curves in Software Development

The prior research on the organizational learning curve has primarily examined the effects of learning curves in the manufacturing context (e.g. Adler & Clark, 1991; Argote et al., 1990; Joskow & Rozanski, 1979; Lieberman, 1984). More recently, some studies have examined the learning curve in the context of the service industry such as in pizza-making and cardiac surgery (e.g. Darr, Argote, & Epple, 1995; Pisano et al., 2001). With the change in the business landscape brought about by the transformation of the industrial organization into a knowledge-based organization (Drucker, 1988), it is important to examine whether the advantages of experience evident in manufacturing tasks are replicated in knowledge work. Unlike the manufacturing task, which is often repetitive (Argote, 1999; Joskow & Rozanski, 1979) or service work, which is frequently scripted (Leidner, 1993), knowledge work tends to entail less routinization and often requires the use of creativity in order to produce knowledge (Ledford, 1995; Schultze, 2000). Hence, one may speculate that the smaller extent of routinization and repetitiveness would result in less learning from experience in knowledge work, because most problems and projects differ across different clients and contexts (Sviokla, 1996). On the other hand, since knowledge work is not limited by the physical constraints in the manufacturing environment such as machinery and raw material constraints (Kemerer, 1998; Yelle, 1979), there may be an even greater potential for organizations to exhibit learning in the context of

knowledge work. Even though there is little repetition of work, organizations can nevertheless gain experience in improving work processes, such as managing of expertise and coordination.

Software systems development is a particular kind of knowledge work. The key inputs required for software development are labor and expertise (Curtis, Krasner, & Iscoe, 1988; Walz, Elam, & Curtis, 1993). Software development is a problem-solving and knowledge-intensive activity (Land, Aurum, & Handzic, 2001) that requires a considerable extent of abstract, technical, and theoretical knowledge (Frenkel, Korczynski, Donoghue, & Shire, 1995; Starbuck, 1992) as well as tacit and experiential knowledge (Sacks, 1994). Although many development projects are different and customized to the requirements of users (Basili & Caldiera, 1995), with each project, the developer gains practice, and builds up an intellectual schema of technical and experiential understanding of the software development task (Sacks, 1994). This renders the software systems development context an interesting setting to examine learning curves.

Learning at Multiple Levels of Analyses

While learning curves have been found to exist at the individual, group, organizational and industry levels of analysis (Argote, 1999), there has not been any study that looks at learning curves at several levels of analysis *simultaneously*. It is important to examine the learning curve at multiple levels of analyses because the evidence of learning at one level (e.g. the individual level) does not necessarily imply that learning exists at a different level of analysis (e.g. the organizational level). Many organizational phenomena inherently involve multiple levels of analysis (Bryk & Raudenbush, 1992; Hofmann, 1997; Rousseau, 1985). Learning, in particular, is a concept that can be applied to different levels of analyses. Learning processes differ at different levels. The types of experience and the impacts of the experience on performance are also expected to differ at different levels. Individual learning is based on individuals' own experience and reflections upon their own experience (Steiner, 1996). Group learning is based

upon interactions between individuals resulting in individuals learning from one another, learning about who knows what (Liang, Moreland, & Argote, 1995; Wegner, 1986), and learning to work together (Edmondson, 2002). Organizational learning is based upon the encoding of inferences from history into routines that guide behavior (Levitt & March, 1988). It results in more structural changes, such as the changes made to the routines of an organization, which include forms, rules, procedures, conventions, strategies, and technologies of the organization.

In order to reveal the mechanisms underlying the learning shown by the organizational learning curve, we develop hypotheses in the next sections concerning three levels of organizational learning in this study – the learning from experience at the individual, group and organizational levels. By identifying the types of experience that constitute learning at different levels of analysis, we examine how individual, group, and organizational experience affects the productivity of the organization.

Learning at the Individual Level in Software Development

The learning curve model has been found to apply at the individual level in many studies (Mazur & Hastie, 1978), particularly for cognitive tasks. In the case of software development, the task of coding, debugging, designing and testing of software is a complex and difficult cognitive task (Kim & Lerch, 1997) that builds upon practical experience (Land et al., 2001). While there are general principles guiding these tasks, every instance of these activities is relatively unique because of the contextual differences (e.g. tools used, domain area, application under development and project characteristics), but the success of certain practices (Humphrey, 1995), or the lessons from prior mistakes (Prechelt, 2001) allow developers to learn from their experience. Several studies have examined the impact of individual experience on the productivity of individual software developers. Jeffery (1987) found that the impact of experience (measured by the number of years working in software development) on individual

productivity is significant. Mockus & Weiss (2001) conducted a trend analysis of the productivity of a group of 50 developers over time and found evidence that the average productivity of individual developers within a project improved over time.

We expect to find significant learning from experience at the individual level by software development personnel. We focus on the experience gained by the individual on-the-job, as it is a key part of the training of individuals in organizations (Sacks, 1994). In addition, we differentiate between the experiences gained by developers working on the same or different software systems. Experience in the development of a software system is expected to positively impact the productivity of developers in work involving the same software system. This is because with experience, developers gain familiarity with the domain of the software system, and increase their understanding of the structure and architecture of the modules and files, as well as the code within the system (Robillard, 1999). Thus, we hypothesize that:

H1a: A developer's prior cumulative experience gained from working on a particular system improves that developer's productivity in the development of that system.

Knowledge Transfer by Individuals in Software Development

Individuals in organizational units can learn not only from their own experience in that unit, but can also potentially benefit from the experience of other units within the same organization if the knowledge and experience are successfully transferred across units. Intra-organizational knowledge transfer across different units is becoming increasingly important in organizations (Argote & Ingram, 2000; Argote, Ingram, Levine, & Moreland, 2000).

Organizations that are able to transfer knowledge effectively from one unit to another are more productive and more likely to survive than those that are less adept at knowledge transfer (Argote et al., 2000; Szulanski, 1996). Knowledge transfer in organizations is "the process through which one unit (e.g., group, department or division) is affected by the experience of

another” (Argote & Ingram, 2000). Knowledge transfer manifests itself through changes in the knowledge or performance of the recipient units. Several studies have examined whether knowledge transfers across work shifts (Epple, Argote, & Devadas, 1991), across organizations in the same franchise (Darr et al., 1995), across organizations in the same industry (Baum & Ingram, 1998; Irwin & Klenow, 1994), and across models of a product (Benkard, 2000). Most of these studies adopt the learning curve approach to study knowledge transfers, by examining the extent to which the performance of one unit is affected by the experience of other units.

In the case of software development, the experience gained from developing one software system can potentially be transferred and applied to the development of other software systems (Collier, DeMarco, & Fearey, 1996; Statz, 1999). Successful software projects or software processes in the development of one system can be replicated, by transferring the knowledge and experience at the root of the successes or failures to the rest of the organization (Basili & Caldiera, 1995). In order to transfer the organization’s experience across different software systems, three types of mechanisms can be used – individual employees, organizational systems and structures (e.g. coordination, technology), and actors in the organization’s environment (e.g. suppliers, competitors) (Argote, 1993; Levin, 2000). Given the dependence of systems development on individual software developers, a key way to transfer the software organization’s experiences across systems is to make use of individual employee rotations so that an individual can transfer his or her experience in one system to another (Conradi & Dyba, 2001). Hence, instead of examining the influence of one unit’s experience on the productivity of another unit, we examine whether the experience of an individual developer in one unit affects his or her productivity in another unit (Allen, 1977). While the understanding of the domain or structure of one software system does not necessarily transfer to the development of another software system, developers’ experience and their general lessons learned about designing and

coding software can be applied to the development of different software systems. In addition, individuals' knowledge gained about organizational processes and usage of tools can also be transferred across systems. We therefore hypothesize that:

H1b: A developer's prior cumulative experience gained from projects on other systems improves that developer's productivity in the development of the current system.

Learning at the Group Level in Software Development

While some of the work in the development of large-scale software systems is accomplished by individual developers and later integrated with the work of others, teams are frequently used in software development due to the size and complexity of systems projects. In projects involving teams of software developers, individuals have to ensure effective teamwork coordination (Herbsleb & Grinter, 1999; Kraut & Streeler, 1995). Brooks (1995) has highlighted the difficulty of coordination and communication between individuals when a team is used in software development. The need for coordination is exacerbated in large-scale software development, where interdependencies abound among individuals (Kraut & Streeler, 1995).

Several studies have examined the question of how teams can improve their performance in software development. For example, Faraj & Sproull (2000) found that expertise coordination has a significant impact on team performance. The mere presence of expertise on a team is insufficient to produce good work; rather, teams must be able to manage their skills and knowledge interdependencies in order to work well together. Espinosa (2002) also found that the team organization of task, team communication, and team cognition play an important role in software development. As collaborators interact over time and develop familiarity with their task, they develop organized common knowledge about tasks, goals and strategies that facilitates their work. This shared knowledge can thus help collaborators to manage tasks and member interdependencies more effectively. Research on groups has also shown that experience working

with one another leads individuals to develop transactive memory – knowledge about who is good in doing what – thus improving team performance (Liang et al., 1995; Wegner, 1986). Katz (1982)'s work on group longevity also shows that groups that have members with less experience working with one another exhibit lower performance. Hence, consistent with prior research (Espinosa, 2002), beyond individual level learning, we expect that as software developers work together on systems development projects, team members will learn about coordination, will develop shared cognition, will build transactive memory, and will learn about effective expertise coordination mechanisms. We therefore hypothesize that:

H2: In software development projects involving more than one individual, the developers' prior experience of working with one another will improve group productivity in the current system, controlling for the developers' prior cumulative experience with the system and with other systems.

Learning at the Organizational Level in Software Development

Since software development is a knowledge-intensive activity, and the key input required is expertise (Faraj & Sproull, 2000), the ability of the organization to learn from its experience is highly dependent on its employees. This could limit the extent to which the organization learns from its experience, as software professionals have traditionally been known for their high turnover rates (Meta Group, 1999; Sribar, Schafer, Cearley, Jack, & Zachmann, 2001). If software organizations depend too much on individuals to accumulate knowledge and to deliver on software projects, there could be problems of knowledge depreciation when these individuals leave and take this knowledge with them. Moreover, with the fast pace of change, the time pressures, and the keen competition faced by organizations developing software systems, there may be a tendency for many organizations to focus more on fire-fighting their more immediate problems than on systematic learning from one project to another.

On the other hand, many tools and processes (Paulk, Curtis, Chrissis, & Weber, 1993; Software Engineering Institute, 1995) can be put in place by organizations to ensure that the knowledge of individuals is retained in the routines (Levitt & March, 1988), repositories and artifacts of the organization even after the individuals leave. This ensures that the software production within an organization does not completely depend on the continued tenure of one or a few developers. Moreover, although each product that the organization produces is different, the organization still has the opportunity to learn from the successes and failures in developing different products (Statz, 1999). Hence, whether organizations involved in software development learn from their experience remains an open question that we seek to examine in this study.

Many software organizations adopt the approach of evolving and enhancing a software system (e.g., MacCormack, Verganti, & Iansiti, 2001). These organizations introduce different releases of the software to the market at different times, with incremental features and improvements made to each release of the software. Each release constitutes a major production effort. Within a single release, each task such as budget estimation, project scheduling, requirements gathering, and software design is conducted only once, with limited repetition. Across releases, on the other hand, organizations can reuse, not only software components, but also useful experiences like: software processes and tools that have proven to be successful (Basili & Caldiera, 1995; Ellmer, Merkl, Quirchmayr, & Tjoa, 1996); project management skills that have contributed to projects completed on time and within budget (Collier et al., 1996; Statz, 1999); and management of team culture that has contributed to the formation of top performing teams (Land et al., 2001). For organizations to learn about the effectiveness of their routines and software processes and to identify the need for new processes and routines, we expect organizations to exhibit learning across software releases, rather than across units of work within a software release, as organizational experience is expected to have a longer-term impact than

individual or group experience. Hence, we expect the number of system releases completed by the organization to form the basis of the organizational learning. That is:

H3: The prior experience of the organization, defined in terms of the number of software releases completed prior to the start of a unit of work in the current system, has a positive impact on the productivity of the development of the current system.

III. DATA AND METHODS

To test the above hypotheses in the software development context, we examine an organization conducting large-scale software development in the telecommunications industry. We focused on a flagship product produced by the organization, a telecommunications software switch containing over 80 million lines of code (written in the C programming language). The product is composed of 63 major systems and 27 smaller systems. Each system is produced by a separate internal organization that varies in size from 30 to about 300 employees. Each department has a director, and several managers. Most developers only work actively on one system at a time, but most have experience working on more than one system.

We collected and analyzed a unique archive covering more than 14 years of software production in the organization that dates from the beginning of the development process for each system. In order to examine whether learning occurs in the development of a system, it is imperative to have data about the system from the beginning of its development process. It is also essential to have information about the characteristics of each unit of work completed on the system. Finally, it is necessary to have data collected over a sufficiently long period such that learning can occur and can be exhibited. The archival data set in this study meets these requirements and hence is well suited to the examination of learning curves in software development. The software production data were extracted from the version control system and change management system used by the organization to store and reconstruct past versions of

program source code. These systems capture a great deal of contextual information about each change made to the program source code and include information about the extent of contribution of each individual, information on who worked together with whom on which projects, as well as information about different aspects of the system, such as the complexity, quality, and size of the system. The systems have the ability to track a group of related changes via a modification request (MR), a set of atomic changes that are made for a specific purpose and are thus semantically related. An MR is similar in concept to a work order, and it is the basic unit of work in this organization. All changes made to the software are done via an MR, which is formally reviewed by a change committee before it is approved and assigned to an individual or a group of individuals.

The conventional form of the learning curve can be expressed by the equation $y_t = a x_{t-1}^{-b}$ where y_t is the cost per unit to produce the x th unit in month t , a is the cost of producing the first unit, x_t is the cumulative number of units produced as of month t , and b is a parameter measuring the rate at which costs are reduced as cumulative output increases. The unit of analysis in the conventional form of the learning curve is the time period, in the above example – the month. There are, however, several problems with using a time period as the unit of analysis in our study. First, unlike manufacturing and service contexts, where each unit of work is rather standard, each unit of work in software development differs in its size, complexity, and even in the expertise required. Using time as the unit of analysis requires aggregation and averaging of size, complexity, and other variables to effectively control for variation in the dependent variable. Moreover, the production period for each MR may be much longer than one month, and it is nearly impossible to specify which aspects of the size, complexity, and other control variables relate to production of each MR in the different months for which an MR is in production. Hence, we adopt the approach of Pisano et al. (2001) and Thornton & Thompson

(2001) by analyzing “unit labor requirements” instead of monthly rates of output, and thus define the MR as the unit of our analysis.

We define the dependent variable to be the effort – number of labor hours (in days) – to produce an MR (Thornton & Thompson, 2001; Yelle, 1979). Effort is a good indication of cost in the case of software development because labor cost makes up most of the variable cost of producing software. Unfortunately, our data do not include effort measures for each MR, but the data do indicate the MRs that each individual was working on at any time period. It is an onerous task for organizations to accurately record the effort of developers at such a fine-grained change level. We thus adopted the effort computation algorithm developed in Graves & Mockus (1998) and Atkins, Ball, Graves, & Mockus (2001) that estimates effort for each MR from information available in a version control system. Graves & Mockus (1998) reported that the results of this estimation have been validated using a developer survey and statistical techniques.

In essence, the Graves & Mockus (1998) algorithm keeps track of the number of MRs that each individual developer works on in each month, and divides each developer’s monthly effort equally among the open MRs as a first estimation. This initial estimation of effort for each MR is then used as the dependent variable measure in our regression analysis. The model is then estimated to provide predicted values of the effort per MR based on the estimated coefficients. This is then used to refine the estimation of effort expended for each MR, and the regression analysis and re-estimation steps are repeated until there is no improvement in the log likelihood estimates in the analyses. In all of our analyses, only one iteration of the cycle was required to produce the effort estimates yielding the optimal log likelihood results¹. Appendix A provides more details about the effort computation algorithm.

¹ Moreover, the initial estimate of effort based on equal division of effort among all of the open MRs for an individual produces consistent results as does the second iteration of the cycle, indicating that the initial estimation

In the rest of this section, we present the analysis strategy and operationalizations of the variables for our analyses and state our a priori expectations about how the different variables will impact the dependent variable. For the analysis, we divided the MR data into two different data sets – data set one includes MRs that involve one individual (“individual MRs”), and data set two includes MRs that involve more than one individual (“group MRs”). This is necessary because different factors affect the productivity of the individual MRs and group MRs. Individuals who work alone do not have to spend time coordinating with others in their team, but instead rely on the use of the version control system to ensure synchronization with the other parts of the system. Group experience is thus not applicable to the analysis of productivity for individual MRs.

Analysis of Data Set One on MRs Involving One Individual

For individual MRs, we test hypotheses 1a, 1b, and 3. Hypothesis 2 is not relevant as it refers to group experience. We modeled the impact of individual experience within the system (H1a), individual experience from other systems (H1b), and organizational experience in the system (H3) on the productivity of each MR (Equation 1).

Equation 1²:

$$\begin{aligned} \text{Ln(Effort Per MR}_{ij}) = & \beta_{0i} + \beta_1 \text{Ln(Individual System Experience}_{ij}) + \beta_2 \text{Ln(Individual Experience in Other} \\ & \text{Systems}_{ij}) + \beta_{3i} \text{Ln(Organizational System Experience}_{ij}) + \beta_4 \text{Time}_{ij} + \beta_5 \text{Ln(Priority}_{ij}) \\ & + \beta_6 \text{Ln(Size}_{ij}) + \beta_7 \text{Ln(Original Size}_{ij}) + \beta_8 \text{Ln(Complexity}_{ij}) + \beta_9 \text{New Development}_{ij} \\ & + u_i + \epsilon_{ij} \end{aligned}$$

To test hypothesis 1a, we examine the impact of individual system cumulative experience on productivity. Individual system experience for MR j is defined as the cumulative number of MRs produced by each individual in system i prior to the start of MR j . To test hypothesis 1b, we

of effort based on an equal division of effort among all open MRs can be sufficient to produce the results of our analyses.

² subscript i = system; subscript j = MR

examine the impact of individual cumulative experience in other systems on the productivity in the current system. Individual experience in other systems for MR j is defined as the cumulative number of MRs produced by each individual in other systems prior to the start of MR j . β_1 and β_2 constitute the learning index from which we can derive the rate at which individuals improve their productivity based on their experience gained from the current system and from other systems respectively (Yelle, 1979). In order to test hypothesis 3, we examine whether the experience gained by the organization from past releases of a software system affects the productivity in the current release. Organizational system experience for MR j is thus defined as the cumulative number of releases the organization has experienced in system i prior to the start of MR j .

Consistent with the learning curve literature, we defined learning to be the change in productivity of individuals, groups, or organizations with increasing experience. It is with the development of insights, knowledge, and associations between past actions and the effectiveness of those actions, that individuals, groups, and organizations are able to learn from experience (Edmondson, 2002). We need to differentiate between learning from experience, and adaptation that may be reflected in changes in productivity of the organization as a result of adjustments to environmental changes or technological improvements that are independent of the experience of the organization or individuals within the organization (Fiol & Lyles, 1985). In a technology-focused environment and volatile environment faced by the telecommunications industry, it is important for us to control for time. By including a variable for time, we also control for technological changes over time that are independent of the organization's experience (Argote, 1999; Argote et al., 1990). Hence, we include a control for the time at the start of each MR, which is defined as the number of months from the start of the development of the switch.

It is important to be able to control for other variables that could impact the dependent variable in a learning curve analysis. In the case of software development, each unit of work differs in its characteristics, making it more challenging than manufacturing settings where many units of the same product are usually being produced. Hence, we include controls for the characteristics of MRs. Based on previous research on software development (e.g. Banker et al., 1987; Banker, Datar, & Kemerer, 1993; Banker, Davis, & Slaughter, 1998; Banker & Slaughter, 2000; Deephouse, Mukhopadhyay, Goldenson, & Kellner, 1995; Espinosa, 2002), we identified five variables that characterize each MR for work involving one individual.

Size. The effort put into the development of each software unit has been shown to increase with the size of the software unit (Banker et al., 1998; Banker & Slaughter, 1997; Gopal, Mukhopadhyay, & Krishnan, 2002). The size of an MR can be measured by the lines of codes changed (added and deleted) for each MR, and by the number of deltas logged by each individual (Mockus & Weiss, 2000). A delta is a basic unit of software change involving the modification of a single file by a single developer.

OriginalSize. In software modification, it is important to understand the code in an existing system before one can begin making changes to a file, or to existing code. Hence, the greater the amount of code that each individual has to understand before beginning work on an MR, the greater the amount of effort each individual is expected to spend on an MR (Slaughter, 1995). To operationalize the original size of each MR, all the existing files that an individual has to change to implement the MR are identified, and the lines of codes in these existing files that were not written by the individual making the change are considered to be the lines of codes that the individual would have to understand in order to make the change. The sum of these lines of codes is then used as a measure of the original size of each MR.

Complexity. The complexity of the software affects the productivity of software development (Banker et al., 1993; Banker & Slaughter, 2000; Barry, Kemerer, & Slaughter, 2000; Weyuker, 1988). The amount of coupling between files and modules has been stated as a key indication of the complexity of software (Chidamber & Kemerer, 1994). The greater the extent of coupling between different parts of the software, the more difficult it is to maintain the software. Any changes made to one part of the software affect other parts of the software and more effort is required to ensure the correctness of the software. Hence, we operationalize the complexity of the software as the number of files and modules that have to be changed in order to implement a change required by an MR (Espinosa, 2002; Herbsleb & Mockus, 2003).

Priority. The priority of an MR refers to the importance of the MR. This priority is determined by the change committee, and it normally translates to the order in which the MRs are developed. There are four levels of priority, coded into an ordinal scale ranging from 1 to 4: low (1), medium (2), high (3), and emergency (4). The higher the priority of an MR, the more important the work and the greater the amount of effort expected.

New Development. There are two types of changes that can be made to the software – either new feature development, or feature maintenance, where code is being changed or written to maintain an existing feature. We include a variable to indicate whether an MR is a new development. Maintenance of code may require a lot of effort, but in the end, may affect only one line in one file (Atkins et al., 2001). This is because of the need to expend effort to understand the code and identify the problem areas in the code to be rectified. Hence, controlling for the lines of code affected by MR j , we expect that new development will involve smaller amounts of effort per MR compared to code repairs.

The variables measuring size (number of deltas, number of lines of code (LOC) and complexity (number of files affected, number of modules affected) are highly correlated with one

another. Following the procedures used by Espinosa (2002), the logged values of these four variables were reduced to two orthogonal factors using factor scores from principal components factor analysis with varimax rotation. Principal components analysis is commonly used to mitigate collinearity problems (Greene 1997). The total explained variance of the two factors is 82.0%. The first factor explained 58.1% of the variance, it loaded highly on number of deltas (0.942), number of lines of codes (0.786), and number of files affected (0.871), thus its factor score was used as a measure of MR size. The second factor explained an additional 23.9% of the variance, and it loaded highly on number of modules affected (0.996), so its factor score was used as a measure of MR complexity.

Analysis of Data Set Two on MRs Involving Groups

To examine the impact of experience on work involving groups, we examine the MRs that involve more than one individual (Group MRs). Individuals who work on the same MR have to ensure coordination with one another, hence, prior familiarity (Espinosa, 2002) and prior experience working with one another is expected to facilitate group work. In this analysis, we test hypotheses 1a, 1b, 2 and 3 by examining the effects of average individual experience, average shared group experience and organizational experience on the productivity of group MRs. The following equation is estimated (Equation 2):

Equation 2³:

$$\begin{aligned} \text{Ln(Effort Per MR}_{ij}) = & \beta_{0i} + \beta_1 \text{Ln(Average Individual System Experience}_{ij}) + \beta_2 \text{Ln(Average Individual} \\ & \text{Experience in Other Systems}_{ij}) + \beta_3 \text{Ln(Average Shared Experience}_{ij}) \\ & + \beta_{4i} \text{Ln(Organizational System Experience}_{ij}) + \beta_5 \text{Time}_j + \beta_6 \text{Ln(Priority}_{ij}) + \beta_7 \text{Ln(Size}_{ij}) \\ & + \beta_8 \text{Ln(Original Size}_{ij}) + \beta_9 \text{Ln(Complexity}_{ij}) + \beta_{10} \text{Ln(Number of Developers}_{ij}) \\ & + \beta_{11} \text{New Development}_{ij} + u_i + \epsilon_{ij} \end{aligned}$$

³ subscript i = system; subscript j = MR

Similar to the analysis for individual MRs, we define the dependent variable to be the number of days required (effort) to produce MR j , which is operationalized as the sum of the imputed effort expended by all individuals working on MR j . Average individual system experience refers to the average cumulative number of MRs for system i produced by individuals working on MR j , prior to the start of MR j . Average individual experience in other systems refers to the average cumulative number of MRs for other systems produced by individuals working on MR j , prior to the start of MR j . Average shared experience refers to the number of MRs on which both members of a dyad involved in MR j had worked prior to the start of MR j , averaged across all dyads in the team. Finally, similar to the analysis for individual MRs, we define organizational experience as the number of software releases the organization has worked on in system i prior to the start of MR j .

The control variables for each MR are similar to those in the analysis of individual MRs, except for the inclusion of an additional control variable – the number of developers involved in MR j . As the group size increases, we expect more effort to be expended on coordination and on understanding each others' work and progress (Espinosa, 2002; Kraut & Streeler, 1995), thus increasing the effort required for group MRs. The variables operationalizing size (number of individual MRs, number of deltas, number of lines of code), complexity (number of files affected, number of modules affected), and number of developers are highly correlated. Thus, the same procedures were adopted as in the analysis for individual MRs, where the variables were subjected to principal components factor analysis. The total explained variance of the three factors is 82.5%. The first factor (explaining 56.3% of variance) was used to measure team size, as it loaded highly on number of members (0.817), number of deltas (0.773), number of individual MRs (0.773) and number of files affected (0.737). The second factor (explaining 13.5% of variance) was used to measure MR complexity, as it loaded highly on number of

modules affected (0.921). The third factor (explaining 12.7% of variance) loaded highly on number of lines of code (0.967), so its factor score was used as a measure of MR size.

IV. ESTIMATION AND RESULTS

Estimation

Due to the hierarchical and nested nature of the data collected for our analysis, where each MR represents a repeated measure of productivity for a system, we estimated our model using a hierarchical linear modeling approach⁴. We estimated our models using the restricted maximum likelihood estimation method⁵ implemented with a Newton-Raphson algorithm (Lindstrom & Bates, 1988) in the SAS statistical package using the PROC MIXED procedure (SAS Institute, 2000; Singer, 2002; Wallace & Green, 2002). The procedure makes use of specifications of fixed and random effects to analyze the longitudinal production data that are nested within systems. The hierarchical linear model provides great flexibility as both the number of observations per system and the intervals among the observations across time may vary. The time variable may also be continuous rather than a fixed set of points. All of these features make the method a suitable tool to analyze learning at different levels of analysis (individual, group and organization) using software production data.

The use of hierarchical linear modeling also allows the modeling of the within-system error covariance structure. Three within-system covariance structures were evaluated – independent observations (covariance structure = $\sigma^2 I$), first-order autoregressive (covariance structure = $\sigma^2 \rho^{|i-j|}$), and spatial power (covariance structure = $\sigma^2 \rho^{d_{ij}}$) (Jennrich & Schluchter, 1986; SAS Institute, 2000; Zimmerman & Harville, 1991). The independent observations error

⁴ Hierarchical linear modeling (HLM) is described in (Bryk & Raudenbush, 1992). Hoffman (1997) provides more information about applying HLM in the organizational behavior and organizational theory studies. Ang, Slaughter & Ng (2002) is an example of the application of HLM in the Information Systems area.

⁵ The analysis provides the same results with the maximum likelihood method (also implemented with a Newton-Raphson algorithm)

structure was chosen as a base case for comparison, and the other two error structures were chosen because longitudinal data can exhibit autocorrelation across time. The first-order autoregressive structure is commonly used to correct for autocorrelation, but it assumes that the time interval between observations are equal. We thus examined the spatial power error structure, which has a similar structure to the first-order autoregressive structure, except it assumes a reduction in correlation in relation to the power of the distance between time points. The analyses for equations 1 and 2 were re-run with the three error structures, and consistent results were obtained⁶. The results are thus presented for the independent observation error structure, as the analyses can be conducted with the full data set for this error structure.

Tables 1 and 2 present the descriptive statistics and inter-correlations for the variables used in the analysis of individual MRs and the results of the estimation for individual MRs, respectively. First, we established that there is significant variation in the productivity among individual MRs to be explained (Ang, Slaughter, & Ng, 2002; Bryk & Raudenbush, 1992) by estimating only an intercept term in the model (Model 1 in Table 2). The results indicate that 78.6% of the variance is within systems, while 21.4% of the variance is between systems. Models 2 through 6 were estimated to evaluate the determinants of productivity and the proportion of variance explained from adding each set of determinants to the models was calculated (Ang et al., 2002; Bryk & Raudenbush, 1992). We examined the significance of the incremental variance explained by examining the differences between the deviance statistics (Δdev) for each pair of nested models where deviance statistic is twice the negative log-likelihood, and has a chi-square distribution with the difference in number of parameters between

⁶ The analysis for individual MRs (equation 1), however, was unable to be conducted for the auto-regressive and spatial power error structures because of the huge demand on computational resources due to the large number of data points. Hence, 2000 data points from each system were randomly selected for this error covariance structure sensitivity analysis.

models to be estimated as the degrees of freedom (Kreft & de Leeuw, 1998). The incremental variances explained by models 2 through 6 are significant.

[Insert Tables 1 and 2 Here]

Table 3 presents descriptive statistics and inter-correlations for the variables used in the analysis of group MRs, while the results of the hierarchical linear estimation are presented in Table 4. Similar to the above analysis, we first established that there is significant variation in the productivity among group MRs to be explained, and note that 82.0% of the variance is within systems, while 18.0% of the variance is between systems. Models 2 through 7 were estimated to evaluate the proportion of variance explained from adding each set of determinants to the models. All the incremental variances explained by models 2 through 7 are significant.

[Insert Tables 3 and 4 Here]

Hypothesis Tests

Hypothesis 1a posited a positive relationship between individual cumulative system experience and productivity while hypothesis 1b posited a positive relationship between individual cumulative other system experience and productivity. Both hypotheses are supported as both individual experience variables significantly decreased the labor hours required per MR (Model 4 in both tables 2 and 4), and adding the individual experience variables explained significant incremental variance in the productivity of individual MRs ($\Delta dev = 27107$, $df=2$, $p<.001$) and the productivity of group MRs ($\Delta dev = 579$, $df=2$, $p<.001$). The coefficients (in Table 2) indicate that when individuals are working alone on an MR, a 1% increase in individual experience in the current system prior to the start of an MR is associated with a 0.043% decrease in effort for developing the MR. Similarly, a 1% increase in individual experience in other systems prior to the start of an MR is associated with a 0.047% decrease in effort for the MR.

Hypothesis 2 posited a positive relationship between cumulative shared experience and group productivity. This hypothesis is also supported as shared experience significantly decreases the effort required for each MR, and adding shared experience to Model 5 in table 4 explained significant incremental variance in the productivity of group MRs ($\Delta dev = 503$, $df=1$, $p<.001$). The coefficient of the group shared experience indicates that when individuals are working with others in developing an MR, a 1% increase in the experience of working with one another prior to the start of an MR is associated with a .019% decrease in effort for the MR. This result is consistent with findings in related studies (Espinosa 2002).

Hypothesis 3 posited a positive relationship between cumulative organizational system experience and productivity. This hypothesis is not supported because instead of decreasing the labor hours for each MR, organizational system experience increases the labor hours required for each MR. Adding cumulative organizational system experience to Model 5 in table 2 explained significant incremental variance in the productivity of individual MRs ($\Delta dev = 634$, $df=1$, $p<.001$). Specifying organizational system experience as a random effect for each system in Model 6 (table 2) explains additional incremental variance in the productivity of individual MRs ($\Delta dev = 5124$, $df=89$, $p<.001$), showing that there is significant variation in the impact of organizational system experience on productivity among systems for individual MRs. Adding cumulative organizational system experience to Model 6 in table 4 also explained significant incremental variance in the productivity of group MRs ($\Delta dev = 105$, $df=1$, $p<.001$). In both models, Δdev is much smaller when adding organizational experience, compared to adding the individual experience variables. Specifying organizational system experience as a random effect for each system in Model 7 (table 4) also explains additional incremental variance in the productivity of individual MRs ($\Delta dev = 196$, $df=89$, $p<.001$), showing that there is significant variation in the impact of organizational system experience on productivity among systems for

group MRs. The coefficient of the organizational experience indicates that a 1% increase in the experience of the organization (in terms of number of releases in a system) prior to the start of an MR is associated with a 0.069% (0.053%) increase in effort for the MR for individual MRs (group MRs). Finally, with regards to the control variables, the coefficients of all the control variables are significant and in the expected directions for both sets of analyses.

V. DISCUSSION

Our empirical analysis confirms hypotheses 1a, 1b and 2, and reveals interesting findings for hypothesis 3. Consistent with hypothesis 1a, we find that individuals' prior experience with a system improves their productivity with that same system. In support of hypothesis 1b, we also find that individuals' prior experience in one system transfers to other systems to improve their productivity in work on the other systems. This implies that individuals are effective in transferring their experiences across different systems, confirming that people can be used as an effective mechanism to transfer knowledge across systems in software development (Allen, 1977). The proportion of within-system variance explained for each of the models and the deviance difference (in both Tables 2 and 4) also shows that the individual experience variables appear to have the most significant explanatory power among all of the experience variables. This highlights the importance of individual experience in affecting the productivity of a software organization. In addition, our results also provided support for hypothesis 2, showing that prior experience working with one another significantly improved the productivity of developers working together on an MR. This finding highlights the benefits of putting individuals who have experience working with one another on the same team in software development.

On the other hand, we find that hypothesis 3 is not supported. Our results show that on average, there is no learning from organizational experience across system releases in the

organization under study. However there is significant variance in the coefficients of organizational experience across different systems. Figure 1 shows the intercepts and coefficients of the organizational experience variable for a sample of 7 systems. This figure illustrates that the rates of learning from organizational experience differ significantly across systems.

These results imply that in the context of software development, while individuals and groups appear to learn from their experience, organization level learning is not necessarily a given. One explanation for the lack of organization level learning may be the lack of formal processes introduced within the organization. This organization does not have a rating according to the Capability Maturity Model (CMM) assessment scale (Paulk et al., 1993; Software Engineering Institute, 1995). The CMM for Software developed by the Software Engineering Institute is a framework that describes the key elements of an effective software process. The CMM incorporates a number of organizational learning mechanisms that help a software organization to improve an ad hoc, immature process to a mature, disciplined one. As the organization in this study has not adopted the CMM or any similar program, most of the organizational units lack formal mechanisms such as the use of project post-mortems and software metrics that can help them to learn across system releases.

In sum, we find that there is much variation in the learning rates across different systems, with many systems exhibiting no improvement in productivity with greater organizational experience. This is controlling for the impact of individual and group experience on the productivity in each system. While we offer some potential explanations about why this lack of learning at the organizational level occurred, it would be useful to more systematically examine why some systems exhibited improvement in productivity with more organizational experience, while others did not. This is an area for further research that we plan to examine in future studies.

VI. CONCLUSION

In this study, we find that there is learning from experience in software development, which is one important type of knowledge work. Our findings indicate that while there is some group level learning in software development, most of the learning takes place at the individual level. In the organization that we study, learning at the organization level across system releases is limited, and varies across different systems. This has significant implications for software development organizations. Given the extent of learning at the individual level, organizations need to invest in mechanisms that facilitate learning and take steps to ensure that there is knowledge transfer and sharing among individuals. Organizations should also seek to improve their software processes, both formal and informal, to allow learning from the successes or failures of past software releases and projects.

Our study contributes to the organizational learning literature in two ways. First, we apply the learning curve to knowledge work and specifically to the context of software development and establish that the learning curve is salient in this context. Second, we discern the improvement in productivity that is attributable to experience at the individual, group and organizational levels. Our results show that it is important to differentiate between learning at different levels as there can be learning at some levels (in this case, individual and group) but not at other levels (in this case, organization).

Our study also contributes to the information systems literature. First, we have identified the types of experience that matter at different levels of analysis in the software development context. We identified individual cumulative units of work as the key basis of experience for the individual, average cumulative units of work produced with other team members as the key basis of experience for a team, and cumulative number of software releases as the key basis of experience for the organization. Second, by applying the learning curve to the software

development context, we introduce an objective way of measuring the extent of learning in individuals, teams and organizations engaged in systems development. Finally, our results highlight that learning at the organizational level in the context of software development was lacking in the organization under study. An implication is that software managers should put a greater emphasis on implementing the necessary knowledge transfer mechanisms such as software processes, routines, and project post-mortems to ensure that the organization is able to learn from and leverage experience across system projects. Future research can examine in greater detail the factors affecting organizational learning in software development and the mechanisms through which software organizations can improve their organizational learning.

VII. REFERENCES

- Adler, P. S., & Clark, K. B. 1991. Behind the learning curve: A sketch of the learning process. *Management Science*, 37(3).
- Allen, T. J. 1977. *Managing the flow of technology: Technology transfer and the dissemination of technological information within the R&D organization*. Cambridge, MA: MIT Press.
- Ang, S., Slaughter, S. A., & Ng, K. Y. 2002. Human capital and institutional determinants of information technology compensation: Modeling multi-level and cross-level interactions. *Management Science*, 48(11): 1427-1445.
- Argote, L. 1993. Group and Organizational Learning-Curves - Individual, System and Environmental Components. *British Journal of Social Psychology*, 32: 31-51.
- Argote, L. 1999. *Organizational learning: Creating, retaining and transferring knowledge*. Norwell, MA: Kluwer.
- Argote, L., Beckman, S. L., & Epple, D. 1990. The persistence and transfer of learning in industrial settings. *Management Science*, 36: 140-154.
- Argote, L., & Ingram, P. 2000. Knowledge transfer: A basis for competitive advantage in firms. *Organizational Behavior and Human Decision Processes*, 82(1): 150-169.

- Argote, L., Ingram, P., Levine, J. M., & Moreland, R. L. 2000. Knowledge transfer in organizations: Learning from the experience of others. *Organizational Behavior and Human Decision Processes*, 82(1): 1-8.
- Atkins, D., Ball, T., Graves, T., & Mockus, A. 2001. Using version control data to evaluate the impact of software tools: A case study of the version editor. *IEEE Transactions on Software Engineering*, 28(7): 625-637.
- Banker, R. D., Datar, S. M., & Kemerer, C. 1987. *Factors affecting software maintenance productivity: an exploratory study*. Paper presented at the 8th International Conference on Information Systems.
- Banker, R. D., Datar, S. M., & Kemerer, C. 1993. Software complexity and software maintenance costs. *Communications of the ACM*, 36(11): 81-94.
- Banker, R. D., Davis, G. B., & Slaughter, S. A. 1998. Software Development Practices, Software Complexity, and Software Maintenance Performance: A Field Study. *Management Science*, 44(4): 433-450.
- Banker, R. D., & Slaughter, S. A. 1997. A Field Study of Scale Economies in Software Maintenance. *Management Science*, 43(12): 1709-1725.
- Banker, R. D., & Slaughter, S. A. 2000. The Moderating Effects of Structure on Volatility and Complexity in Software Enhancement. *Information Systems Research*, 11(3): 219-240.
- Barry, E. J., Kemerer, C. F., & Slaughter, S. A. 2000. A Multi-Dimensional Measurement of Software Volatility. *GSIA Working Paper*.
- Basili, V. R., & Caldiera, G. 1995. Improve software quality by reusing knowledge and experience. *Sloan Management Review*, 37(1): 55-64.
- Baum, J. A. C., & Ingram, P. 1998. Survival-enhancing learning in the Manhattan hotel industry, 1898-1980. *Management Science*, 44: 996-1016.
- Benkard, C. L. 2000. Learning and forgetting: The dynamics of aircraft production. *American Economic Review*, 90: 1034-1055.
- Brooks, F. P. 1995. *The Mythical man month: Essays on software engineering* (Anniversary ed.). Reading, M.A.: Prentice-Hall.
- Bryk, A. S., & Raudenbush, S. W. 1992. *Hierarchical linear models: Applications and data analysis methods*. Newbury Park: Sage.
- Chidamber, S. R., & Kemerer, C. F. 1994. A Metrics Suite for Object Oriented Design. *IEEE Transactions on Software Engineering*, 20(6): 476-493.

- Collier, B., DeMarco, T., & Fearey, P. 1996. A defined process for project postmortem review. *IEEE Software*, 13(4): 65-72.
- Conradi, R., & Dyba, T. 2001. *An empirical study on the utility of formal routines to transfer knowledge and experience*. Paper presented at the 8th European software engineering conference held jointly with 9th ACM SIGSOFT symposium on Foundations of software engineering.
- Curtis, B., Krasner, H., & Iscoe, N. 1988. A field study of the software design process for large systems. *Communications of the ACM*, 31(11): 1268-1286.
- Darr, E. D., Argote, L., & Eppele, D. 1995. The acquisition, transfer and depreciation of knowledge in service organizations: Productivity in franchises. *Management Science*, 41: 1750-1762.
- Deephouse, C., Mukhopadhyay, T., Goldenson, D. R., & Kellner, M. I. 1995. Software Processes and Project Performance. *Journal of Management Information Systems*, 12(3): 185-203.
- Drucker, P. F. 1988. The Coming of the New Organization. *Harvard Business Review*, 66(1): 45-53.
- Edmondson, A. C. 2002. The local and variegated nature of learning in organizations: A group-level perspective. *Organization Science*, 13(2): 128-146.
- Ellmer, E., Merkl, D., Quirchmayr, G., & Tjoa, A. M. 1996. *Process model reuse to promote organizational learning in software development*. Paper presented at the 20th International Computer Software and Applications Conference, Seoul, South Korea.
- Eppele, D., & Argote, L. 1996. An empirical investigation of the microstructure of knowledge acquisition and transfer through learning by doing. *Operations Research*, 44(1): 77-86.
- Eppele, D., Argote, L., & Devadas, R. 1991. Organizational learning curves: A method for investigating intra-plant transfer of knowledge acquired through learning by doing. *Organization Science*, 2(1): 58-70.
- Espinosa, J. A. 2002. *Shared mental models and coordination in large-scale, distributed software development*. Unpublished Doctoral Dissertation, Carnegie Mellon University, Pittsburgh.
- Faraj, S., & Sproull, L. 2000. Coordinating expertise in software development teams. *Management Science*, 46(1554-1568).
- Fiol, C. M., & Lyles, M. A. 1985. Organizational Learning. *Academy of Management Review*, 10(4): 803-813.

- Frenkel, S., Korczynski, M., Donoghue, L., & Shire, K. 1995. Re-constituting work: Trends towards knowledge work and information-normative control. *Work, Employment and Society*, 9(4): 773-796.
- Garvin, D. 2000. *Learning in action*. Boston, MA: Harvard Business School Press.
- Gopal, A., Mukhopadhyay, T., & Krishnan, M. S. 2002. The role of software processes and communication in offshore software development. *Communications of the ACM*, 45(4): 193-200.
- Graves, T., & Mockus, A. 1998. *Inferring change effort from configuration management databases*. Paper presented at the Metrics 98: Fifth International Symposium on Software Metrics, Bethesda, Maryland.
- Herbsleb, J. D., & Grinter, R. E. 1999. Architectures, coordination and distance: Conway's law and beyond. *IEEE Software*, 16(5): 63-70.
- Herbsleb, J. D., & Mockus, A. 2003. An Empirical Study of Speed and Communication in Globally-Distributed Software Development. *Forthcoming in IEEE Transactions on Software Engineering*.
- Hofmann, D. A. 1997. An overview of the logic and rationale of heirarchical linear models. *Journal of Management*, 23(6): 723-744.
- Huber, G. P. 1991. Organizational learning: The contributing processes and the literatures. *Organization Science*, 2(1): 88-115.
- Humphrey, W. 1995. *A discipline for software engineering*. MA: Addison-Wesley.
- Irwin, D. A., & Klenow, P. J. 1994. Learning-by-doing spillovers in the semiconductor industry. *Journal of Political Economy*, 102(6): 1200-1227.
- Jeffery, D. R. 1987. A software development productivity model for MIS Environments. *Journal of Systems and Software*, 7(2): 115.
- Jennrich, R. I., & Schluchter, M. D. 1986. Unbalanced Repeated-Measures Models with Structured Covariance Matrices. *Biometrics*, 42: 805 -820.
- Joskow, P. L., & Rozanski, G. A. 1979. The effects of learning by doing on nuclear plant operating reliability. *The Review of Economics and Statistics*, 61(2): 161-168.
- Katz, R. 1982. The effects of group longevity on project communication and performance. *Administrative Science Quarterly*, 27: 81-104.
- Kellner, M. I., Madachy, R. J., & Raffo, D. M. 1999. Software process simulation modeling: Why? What? How? *Journal of Systems and Software*, 46(2/3): 1-18.

- Kemerer, C. F. 1998. Progress, Obstacles, and Opportunities in Software Engineering Economics. *Communications of the ACM*, 41(8): 63-66.
- Kim, J., & Lerch, F. J. 1997. Why is programming (sometimes) so difficult? Programming as scientific discovery in multiple problem spaces. *Information Systems Research*, 8(1): 25-51.
- Kraut, R. E., & Streeler, L. A. 1995. Coordination in Software Development. *Communications of the ACM*, 38(3): 69-81.
- Kreft, I., & de Leeuw. 1998. *Introducing multilevel modeling*. Thousand Oaks, CA: Sage.
- Land, L., Aurum, A., & Handzic, M. 2001. *Capturing implicit software engineering knowledge*. Paper presented at the 13th Australian Software Engineering Conference, Canberra, Australia.
- Ledford, G. E. J. 1995. Paying for the skills, knowledge, and competencies of knowledge workers. *Compensation and Benefits Review*, 27(4): 55-62.
- Leidner, R. 1993. *Fast food, fast talk: Service work and the routinization of everyday life*. Berkeley: University of California Press.
- Levin, D. Z. 2000. Organizational learning and the transfer of knowledge: An investigation of quality improvement. *Organization Science*, 11(6): 630-647.
- Levitt, B., & March, J. G. 1988. Organizational Learning. *Annual Review of Sociology*, 14: 319-340.
- Liang, D. W., Moreland, R., & Argote, L. 1995. Group versus individual training and group performance: The mediating role of transactive memory. *Personality and Social Psychology Bulletin*, 21: 384-393.
- Lieberman, M. B. 1984. The learning curve and pricing in the chemical processing industries. *The RAND Journal of Economics*, 15(2): 213-228.
- Lindstrom, M. J., & Bates, D. M. 1988. Newton-Raphson and EM Algorithms for Linear Mixed-Effects Models for Repeated-Measures Data. *Journal of the American Statistical Association*, 83: 1014-1022.
- MacCormack, A., Verganti, R., & Iansiti, M. 2001. Developing products on "Internet time": The anatomy of a flexible development process. *Management Science*, 47(1): 133-150.
- Mann, C. C. 2002. Why software is so bad. *Technology Review*. July/August 2002: 33-38
- Mazur, J. E., & Hastie, R. 1978. Learning as accumulation: A reexamination of the learning curve. *Psychological Bulletin*, 85: 1256-1274.

- Meta Group. 1999. *Meta study found demand for IT workers high after Y2K; Found current turnover 11 to 20%*. Retrieved 17 Jun, 2001 from <http://www.ita.org/workforce/resources/articles.htm#4>
- Mockus, A., & Weiss, D. M. 2000. Predicting risk of software changes. *Bell Labs Technical Journal*, 5(2): 169-180.
- Mockus, A., & Weiss, D. M. 2001. Globalization by chunking: A quantitative approach. *IEEE Software*, 18(2): 30-37.
- Nonaka, I. 1991. The knowledge-creating company. *Harvard Business Review*, 69(6): 96-104.
- Paulk, M. C., Curtis, B., Chrissis, M. B., & Weber, C. V. 1993. Capability Maturity Model, Version 1.1. *IEEE Software*, 10(4): 18-27.
- Pisano, G. P., Bohmer, R. M. J., & Edmondson, A. C. 2001. Organizational differences in rates of learning: Evidence from the adoption of minimally invasive cardiac surgery. *Management Science*, 47(6): 752-768.
- Prechelt, L. 2001. Accelerating learning from experience: Avoiding defects faster. *IEEE Software*, 18(6): 56-61.
- Robillard, P. N. 1999. The role of knowledge in software development. *Communications of the ACM*, 42(1): 87-92.
- Rousseau, D. M. 1985. Issues of level in organizational research: Multi-level and cross-level perspectives. *Research in Organizational Behavior*, 7: 1-37.
- Sacks, M. 1994. *On-the-job learning in the software industry*. Westport, Connecticut: Quorum Books.
- SAS Institute. 2000. *SAS OnlineDoc®*, Version 8. Retrieved from
- Schultze, U. 2000. A confessional account of an ethnography about knowledge work. *MIS Quarterly*, 24(1): 3-41.
- Singer, J. D. 2002. Fitting individual growth models using SAS PROC MIXED. In D. S. Moskowitz, & S. L. Hershberger (Eds.), *Modeling intraindividual variability with repeated measures data: Methods and applications*. Mahwah, N.J.: L. Erlbaum Associates.
- Slaughter, S. A. 1995. *Software development practices and software maintenance performance: A field study*. Unpublished Doctoral Dissertation, University of Minnesota, Minnesota.
- Software Engineering Institute, C. M. U. 1995. *The Capability Maturity Model: Guidelines for Improving the Software Process*. Reading, MA: Addison-Wesley.

- Sribar, V., Schafer, M., Cearley, D., Jack, G., & Zachmann, W. 2001. *Commentary: Jobs outpace workers*. Retrieved 16 Jun, 2001 from <http://news.cnet.com/news/0-1007-201-5531886-0.html>
- Starbuck, W. 1992. Learning by knowledge-intensive firms. *Journal of Management Studies*, 29(6): 713-740.
- Statz, J. 1999. Leverage your lessons. *IEEE Software*, 16(2): 30-32.
- Steiner, G. 1996. *Learning: Nineteen scenarios from everyday life*. Cambridge, U.K.: Cambridge Univeristy Press.
- Sviokla, J. J. 1996. Knowledge workers and radically new technology. *Sloan Management Review*, 37(4): 25-40.
- Szulanski, G. 1996. Exploring internal stickiness: Impediments to the transfer of best practice within the firm. *Strategic Management Journal*, 17: 27-43.
- Thornton, R. A., & Thompson, P. 2001. Learning from experience and learning from others: An exploration of learning and spillovers in wartime shipbuilding. *American Economic Review*, 91(5): 1350-1368.
- Wallace, D., & Green, S. B. 2002. Multilevel modeling of longitudinal and functional data. In D. S. Moskowitz, & S. L. Hershberger (Eds.), *Modeling intraindividual variability with repeated measures data: Methods and applications*. Mahwah, N.J.: L. Erlbaum Associates.
- Walz, D. B., Elam, J. J., & Curtis, B. 1993. Inside a software design team: Knowledge acquisition, sharing and integration. *Communications of the ACM*, 36(1): 63-77.
- Wegner, D. M. 1986. Transactive memory: A contemporary analysis of the group mind. In B. Mullen, & G. R. Goethals (Eds.), *Theories of Group Behavior*: 185-205. New York: Springer-Verlag.
- Weyuker, E. J. 1988. Evaluating Software Complexity Measures. *IEEE Transactions on Software Engineering*, 14(9): 1357-1365.
- Yelle, L. E. 1979. The learning curve: Historical review and comprehensive survey. *Decision Sciences*, 10: 302-328.
- Zimmerman, D. L., & Harville, D. A. 1991. A Random Field Approach to the Analysis of Field-Plot Experiments and Other Spatial Experiments. *Biometrics*, 47: 223 -239.

Table 1: Correlation Matrix for Individual MRs

Variable	Mean (s.d.)	1	2	3	4	5	6	7	8	9
1. Ln Effort	-2.186 (1.048)									
2. Ln Size	-0.094 (0.931)	0.144								
3. Ln Original File Size	-5.838 (10.047)	0.114	0.192							
4. Ln Complexity	-0.006 (1.003)	0.114	-0.022	0.072						
5. New Development	0.415 (0.493)	-0.183	0.124	0.055	-0.007					
6. Priority	2.167 (1.037)	0.158	-0.131	-0.045	-0.003 ^a	-0.491				
7. Time (in months)	4625 (1539)	-0.111	-0.122	-0.026	0.090	0.041	0.074			
8. Ln Individual System Experience	4.584 (3.183)	-0.158	0.030	-0.323	-0.051	-0.103	0.091	0.072		
9. Ln Individual Other System Experience	4.308 (3.720)	-0.224	-0.057	-0.077	-0.018	0.057	0.039	0.238	0.176	
10. Ln Organizational System Experience	3.127 (1.135)	-0.106	-0.143	-0.012	0.040	0.059	0.083	0.457	0.090	0.264

All correlations with absolute value > 0.004 are significant at $p\text{-value} < .001$;

Number of individual MRs = 524,175;

Table 2: Results for Individual MRs (Dependent Variable: Ln Effort Per MR)

Coefficients (Std Error)	Model 1	Model 2	Model 3	Model 4	Model 5	Model 6
Intercept	-2.186*** (0.001448)	-1.996*** (0.05138)	-1.701*** (0.05291)	-1.577*** (0.04611)	-1.555*** (0.05877)	-1.664*** (0.06659)
Ln Size		0.166*** (0.001508)	0.154*** (0.001509)	0.171*** (0.001479)	0.172*** (0.001478)	0.170*** (0.001476)
Ln Original File Size		0.009*** (0.000137)	0.008*** (0.000137)	0.003*** (0.000141)	0.003*** (0.000141)	0.003*** (0.00014)
Ln Complexity		0.011*** (0.001639)	0.017*** (0.001632)	0.017*** (0.001591)	0.017*** (0.00159)	0.015*** (0.001616)
New Development		-0.281*** (0.003174)	-0.259*** (0.003172)	-0.260*** (0.003101)	-0.262*** (0.003099)	-0.257*** (0.003097)
Priority		0.108*** (0.001506)	0.118*** (0.001505)	0.131*** (0.001469)	0.130*** (0.001468)	0.133*** (0.001469)
Time (in months)			-0.002*** (0.00001)	-0.001*** (0.00001)	-0.002*** (0.00004)	-0.002*** (0.00005)
Ln Individual System Experience				-0.043*** (0.000469)	-0.044*** (0.00047)	-0.043*** (0.000472)
Ln Individual Other System Experience				-0.045*** (0.00039)	-0.046*** (0.000391)	-0.047*** (0.000396)
Ln Organizational Experience					0.079*** (0.003079)	0.069*** (0.01767)
System Random Effects Variance		0.223	0.236	0.177	0.294	0.360
System Ln Organizational Experience Random Effects Variance						0.023
Deviance (-2 Log Likelihood)	1537056	1450703	1445207	1418100	1417466	1412342
Deviance Difference (Δ Dev)		86353***	5496***	27107***	634***	5124***
Proportion of Within- System Variance Explained		15.29%	16.20%	20.38%	20.47%	21.29%

* $p < .05$; ** $p < .01$; *** $p < .001$;

Number of individual MRs = 524,175;

Number of systems= 90;

Table 3: Correlation Matrix for Group MRs

Variables	Mean (s.d.)	1	2	3	4	5	6	7	8	9	10	11
1. Effort	0.098 (1.216)											
2. Original File Size	5.959 (2.192)	0.587										
3. Size	0.008 (0.991)	0.282	0.272									
4. Complexity	-0.011 (1.002)	0.230	0.356	0.005								
5. Team Size	0.017 (0.983)	0.538	0.653	-0.002	0.008							
6. Priority	1.272 (1.270)	-0.081	-0.102	-0.015	-0.065	-0.059						
7. New Development	0.346 (0.476)	0.092	0.205	0.070	0.054	0.240	0.152					
8. Time (in months)	4202 (1599)	-0.142	-0.072	0.014	0.037	-0.057	0.043	0.018				
9. Average Individual System Experience	4.812 (3.494)	-0.093	0.017	-0.141	0.004	0.076	0.040	-0.066	0.134			
10. Average Individual Other System Experience	4.963 (3.892)	-0.171	-0.069	-0.025	-0.080	-0.010	0.184	0.146	0.271	0.151		
11. Average Shared Experience	-5.050 (6.368)	-0.137	0.006	-0.281	-0.051	0.178	0.011	-0.019	0.068	0.278	0.093	
12. Organizational System Experience	3.032 (1.210)	-0.109	-0.082	0.011	-0.020	-0.081	0.074	0.028	0.534	0.266	0.269	0.068

All correlations with absolute value > 0.018 are significant at $p\text{-value} < .001$;

Number of group MRs = 30,569

Table 4: Results for Group MRs (Dependent Variable: Ln Effort Per MR)

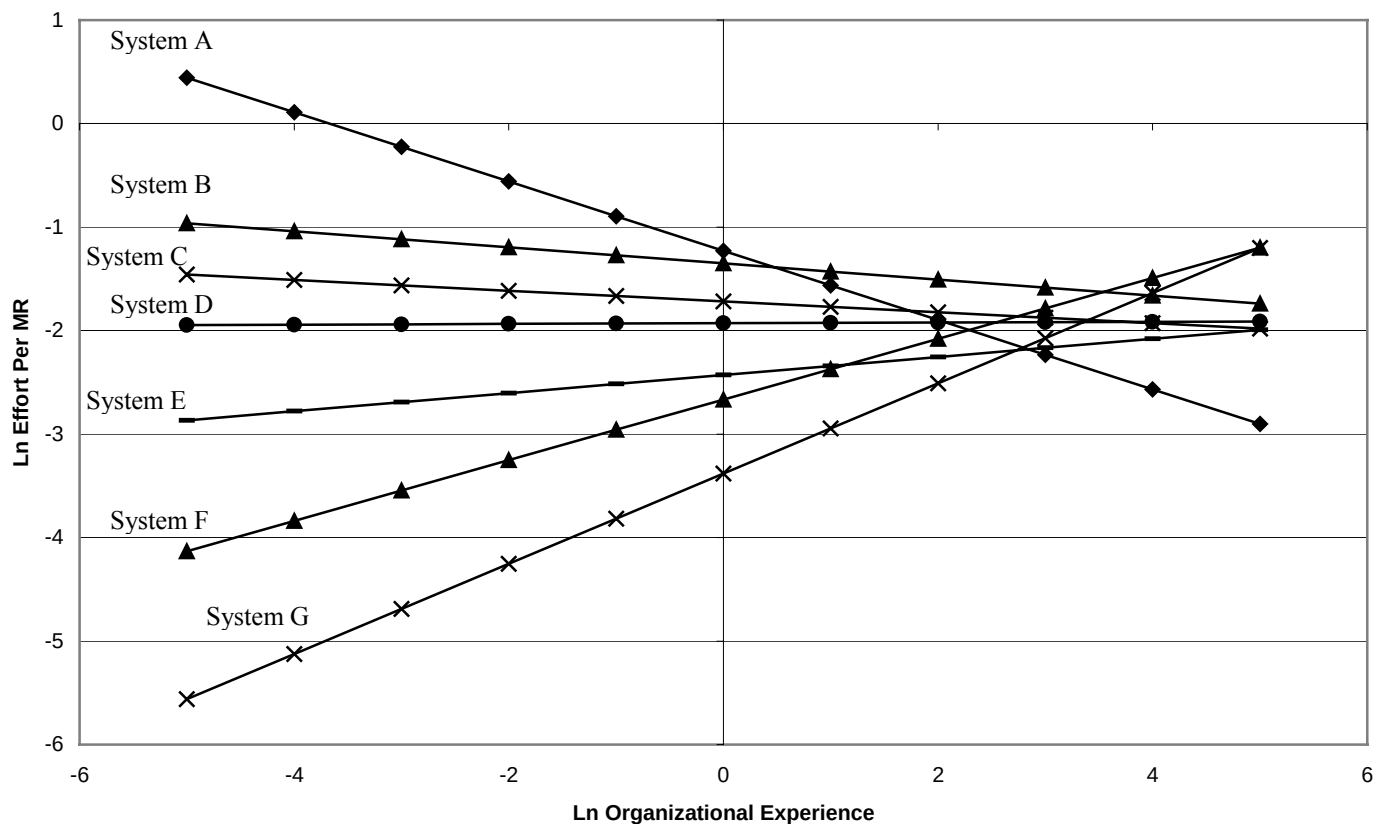
Coefficients (Std Error)	Model 1	Model 2	Model 3	Model 4	Model 5	Model 6	Model 7
Intercept	0.098*** (0.006956)	-0.371*** (0.049400)	0.017 (0.051230)	0.080 (0.050050)	-0.049 (0.049200)	0.014 (0.056950)	-0.049 (0.061210)
Original File Size		0.103*** (0.003688)	0.094*** (0.003668)	0.095*** (0.003633)	0.093*** (0.003604)	0.092*** (0.003599)	0.091*** (0.003595)
Size		0.214*** (0.005570)	0.222*** (0.005523)	0.213*** (0.005500)	0.189*** (0.005557)	0.188*** (0.005545)	0.190*** (0.005533)
Complexity		0.068*** (0.006103)	0.080*** (0.006062)	0.075*** (0.006010)	0.077*** (0.005960)	0.080*** (0.005958)	0.079*** (0.005978)
Team Size		0.554*** (0.007408)	0.558*** (0.007337)	0.561*** (0.007269)	0.583*** (0.007275)	0.590*** (0.007284)	0.592*** (0.007276)
Priority		-0.018*** (0.004089)	-0.015*** (0.004050)	0.001 (0.004065)	0.001 (0.004031)	0.002 (0.004024)	0.005 (0.004021)
New Development		-0.122*** (0.011040)	-0.117*** (0.010940)	-0.098*** (0.010950)	-0.109*** (0.010870)	-0.112*** (0.010850)	-0.109*** (0.010830)
Time (in months)			-0.002*** (0.000098)	-0.002*** (0.000102)	-0.002*** (0.000101)	-0.002*** (0.000126)	-0.002*** (0.000145)
Average Individual System Experience				-0.016*** (0.001514)	-0.010*** (0.001528)	-0.013*** (0.001558)	-0.013*** (0.001563)
Average Individual Other System Experience				-0.029*** (0.001410)	-0.027*** (0.001400)	-0.029*** (0.001407)	-0.031*** (0.001420)
Average Shared Experience					-0.019*** (0.000828)	-0.019*** (0.000826)	-0.019*** (0.000823)
Organizational System Experience						0.066*** (0.006083)	0.053*** (0.011670)
System Random Effects Variance		0.1268	0.1235	0.1154	0.108	0.1681	0.1889
System Experience Random Effects Variance							0.006558
Deviance (-2 Log Likelihood)	98722.6	77045.7	76457	75878.5	75375.7	75270.8	75075
Deviance Difference (Δ Dev)		21677***	589***	579***	503***	105***	196***
Proportion of Within- System Variance Explained		51.24%	52.20%	53.13%	53.90%	54.12%	54.58%

* $p < .05$; ** $p < .01$; *** $p < .001$;

Number of group MRs = 30,569;

Number of systems= 82

Figure 1: Differences in the Impact of Organizational Experience on Effort Per MR
for Individual MRs - A Sample of 7 Systems



Appendix A: Effort computations for Developer Effort

To compute the number of labor hours expended on each MR by each developer, the algorithm adapted from Atkins et al. (2001) and Graves & Mockus (1998) was used. We illustrate the algorithm using an example for a single developer. Developer A worked on MR A in January, and worked on MR A and MR B in February. We assume that each developer spends 1 unit of time per month, an assumption that has been shown to provide similar results as using monthly time sheet data (Atkins et. al ,2001).

Step 1. Divide up the monthly effort equally across all changes open in that month to obtain estimates of total MR efforts for each developer. Developer A spends one unit of effort on MR A in January, 0.5 units of effort on MR A in February, and 0.5 units of effort in February on MR B. The initial estimation of effort is thus 1.5 units of effort for MR A, and 0.5 units of effort for MR B.

Step 2. We fit our regression models (equations 1 & 2) using the initial estimation of effort as the dependent variable measures for developer A. For individual MRs, the predicted values become the new estimates of MR effort. For group MRs, the predicted values of effort for group MRs are allocated to individuals based on the proportion of effort (using estimates from step 1) expended by each individual.

Step 3. Next, we rescale the rows in the imputed monthly MR effort table for each developer so that the new row sums are equal to the regression's fitted values. If the predicted effort for MR A is 2.0, we then impute that developer A spent $(1/1.5 \times 2)$ 1.33 units of effort in January, and 0.67 units of effort in February on MR A. If the predicted value for MR B is 0.5, we then impute that developer B spent 0.5 units of effort in February on MR B.

Step 4. We rescale the columns of the table so that the column sums are equal to the observed monthly efforts for each developer. In our example for developer A, effort for MR A is rescaled to 1.0 in January, and $(0.67/(0.67+0.5) \times 1.0)$ 0.57 in February, while effort for MR B is rescaled to 0.43 in February.

Iterate. The above 4 steps are repeated for each iteration, until convergence, which occurs when the improvement in the error measure in the model fitting step is negligible. For our analysis, the log likelihood measure converged at the 2nd iteration. Hence, the imputed effort used for the 2nd iteration were used as effort measures for the regression analyses.